

Baseline Classifier Evaluation — Bug Prediction Task

Dataset: 1310 labeled file-commit pairs mined from apache/commons-csv via PyDriller (bug-fix vs. clean commits). Train/test split: 80/20, stratified, test set size 262.

Submitted by : **Habibur Rahman Mahin**

Roll: **BSSE1422**

Methodology

Four handcrafted metrics were computed per file from raw Java source using regex-based extraction: Lines of Code (LOC), Cyclomatic Complexity (decision-point count + 1), Variable Count (type-declaration matches), and Comment Density (comment lines / LOC). Features were standardized before training Logistic Regression and SVM; tree-based models used raw feature values.

Results

Model	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	0.491	0.405	0.444	0.499
Decision Tree	0.538	0.489	0.512	0.530
Random Forest	0.537	0.557	0.547	0.568
SVM	0.570	0.435	0.494	0.582
Naive Bayes	0.514	0.290	0.371	0.515

Best F1-Score: Random Forest (0.547)

Best AUC-ROC: SVM (0.582)

Discussion

Random Forest gives the best overall F1-Score, and SVM the best AUC-ROC, but all models sit close to random-guess territory (AUC-ROC 0.50-0.58). This suggests the four handcrafted metrics alone carry weak predictive signal for this dataset - likely because LOC, cyclomatic complexity, and variable count are coarse proxies that don't capture structural or historical risk factors. Naive Bayes performs worst, consistent with its independence assumption being a poor fit when these metrics are correlated (e.g. LOC and cyclomatic complexity typically rise together).

Task 4 (Halstead, CK metrics, process metrics such as code churn and fix history) is expected to add meaningfully more predictive signal, since process metrics in particular are known from prior defect-prediction literature to outperform static code metrics alone.